RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence and Data Science
V Semester B. Tech AIDS

AD23532 - Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 6b: Time Series Data Analysis and Forecast

Name: Nischal **Register Number:** 241801188	**Date:** 09-10-2026
Dataset: Airline Passengers (monthly, 1949-1960)	**Tools:** Python, Pandas, NumPy, Matplotlib, Seaborn, statsmodels, scikit-learn

Learning Objective

To understand and implement the fundamental concepts of time series data analysis and forecasting using Python.

Problem Statement

An airline company keeps monthly records of the number of passengers travelling by air. The company wants to understand historical passenger trends and seasonal patterns and use them to forecast future passenger demand. As a Data Scientist, the task is to analyse the monthly airline passenger data, determine whether the series is stationary, apply suitable transformations where necessary, build an ARIMA forecasting model, predict passenger numbers for the test period and evaluate the accuracy of the forecasts.

Task 1: Download the airline-passengers dataset

In [1]:
```
import itertools, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error
warnings.filterwarnings('ignore'); sns.set_theme(style='whitegrid')

url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv'
df = pd.read_csv(url)                       # downloaded from the Datasets repository
print("Task 1: dataset available:", len(df) > 0)
```

Out [1]:
```
Task 1: dataset available: True
```

Task 2: Read the CSV with a DatetimeIndex

In [2]:
```
df = pd.read_csv(url, parse_dates=['Month'], index_col='Month')
df.index.freq = 'MS'                        # month-start frequency
y = df['Passengers']
print("Task 2: index type:", type(df.index).__name__, "| frequency:", df.index.freqstr)
```

Out [2]:
```
Task 2: index type: DatetimeIndex | frequency: MS
```

> **Observation:** Parsing the Month column and using it as the index gives a DatetimeIndex with monthly (month-start) frequency, which time series models need.

Task 3: Explore the dataset

In [3]:
```
print("Shape:", df.shape, "| Column:", list(df.columns))
print("\nFirst 5 records:"); print(df.head())
print("\nLast 5 records:"); print(df.tail())
print("\nDataset information:"); df.info()
print("\nSummary statistics:"); print(df.describe().round(2))
print("\nMissing values:", df.isnull().sum().sum(), "| Duplicate months:", df.index.duplicated().sum())
print("Period covered:", df.index.min().strftime('%Y-%m'), "to", df.index.max().strftime('%Y-%m'))
print("\nAverage passengers per year:"); print(y.groupby(y.index.year).mean().round(1))
```

Out [3]:
```
Shape: (144, 1) | Column: ['Passengers']

First 5 records:
            Passengers
```

```
Month
1949-01-01         112
1949-02-01         118
1949-03-01         132
1949-04-01         129
1949-05-01         121

Last 5 records:
            Passengers
Month
1960-08-01         606
1960-09-01         508
1960-10-01         461
1960-11-01         390
1960-12-01         432

Dataset information:
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Freq: MS
Data columns (total 1 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Passengers  144 non-null    int64
dtypes: int64(1)
memory usage: 2.2 KB

Summary statistics:
       Passengers
count      144.00
mean       280.30
std        119.97
min        104.00
25%        180.00
50%        265.50
75%        360.50
max        622.00

Missing values: 0 | Duplicate months: 0
Period covered: 1949-01 to 1960-12

Average passengers per year:
Month
1949    126.7
1950    139.7
1951    170.2
1952    197.0
1953    225.0
1954    238.9
1955    284.0
1956    328.2
1957    368.4
1958    381.0
1959    428.3
1960    476.2
```

> **Observation:** The dataset has 144 monthly records (January 1949 to December 1960) in one column, with no missing values and no duplicate months. Passengers range from 104 to 622 with a mean of 280.30. The yearly average rises every year, from 126.7 in 1949 to 476.2 in 1960, so there is a strong upward trend.

Task 4: Plot the series - trend, seasonality and outliers

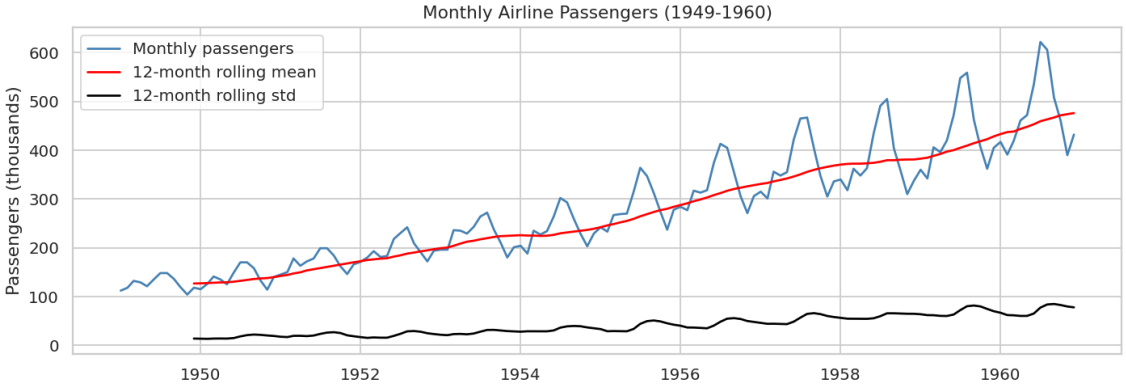
In [4]:

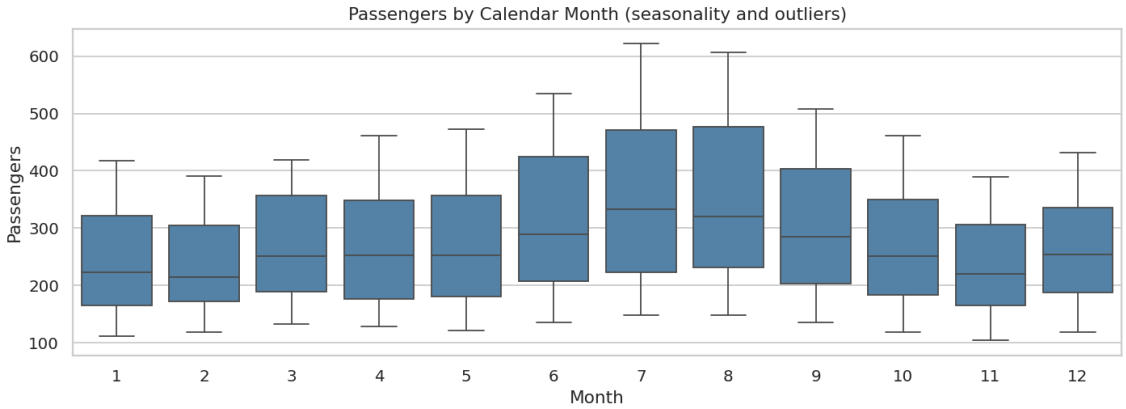
```
fig, ax = plt.subplots(2, 1, figsize=(11, 8))
ax[0].plot(y, color='steelblue', label='Monthly passengers')
ax[0].plot(y.rolling(12).mean(), color='red', label='12-month rolling mean')
ax[0].plot(y.rolling(12).std(), color='black', label='12-month rolling std')
ax[0].set_title('Monthly Airline Passengers (1949-1960)'); ax[0].set_ylabel('Passengers (thousands)'); ax[0].legend()
sns.boxplot(x=y.index.month, y=y.values, color='steelblue', ax=ax[1])
ax[1].set_title('Passengers by Calendar Month (seasonality and outliers)')
ax[1].set_xlabel('Month'); ax[1].set_ylabel('Passengers'); plt.tight_layout(); plt.show()

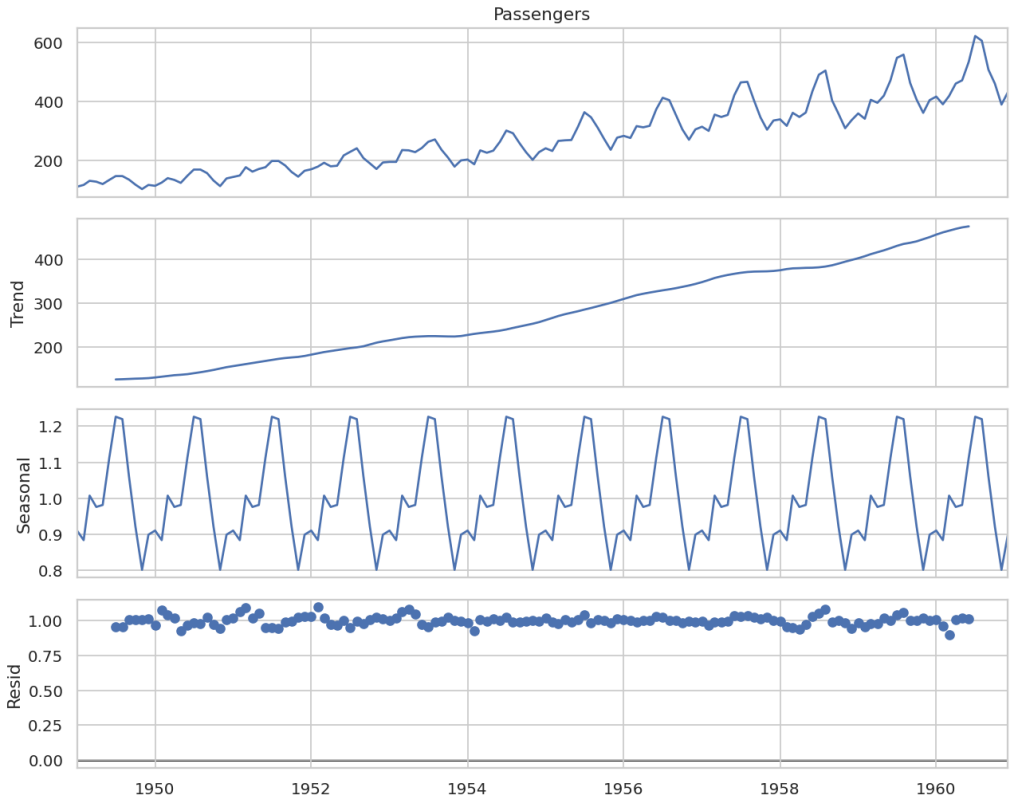
dec = seasonal_decompose(y, model='multiplicative', period=12); dec.plot(); plt.show()
m = y.groupby(y.index.month).mean()
print("Task 4: Peak month on average:", m.idxmax(), "| Lowest month on average:", m.idxmin())
```

Out [4]:

```
Task 4: Peak month on average: 7 | Lowest month on average: 11
```







Observation: The series has a clear upward trend and a repeating yearly pattern: the 12-month rolling mean keeps rising and the rolling standard deviation also grows, so the swings get larger as the level rises. The calendar-month boxplot shows the seasonality, with July (month 7) the busiest month on average and November (month 11) the quietest. No point lies outside the boxplot whiskers, so there are no outliers.

Task 5: Stationarity check with the Augmented Dickey-Fuller (ADF) test

In [5]:

```
def adf_report(series, name):
    r = adfuller(series.dropna(), autolag='AIC')
    print(f"ADF test on {name}")
    print(f"  ADF statistic    : {r[0]:.4f}\n  p-value          : {r[1]:.6f}")
    print(f"  Lags used        : {r[2]}\n  Observations     : {r[3]}")
    for k, v in r[4].items(): print(f"  Critical value {k:>3}: {v:.4f}")
    print("  Result           :", "STATIONARY (p < 0.05, reject H0)" if r[1] < 0.05
          else "NON-STATIONARY (p >= 0.05, cannot reject H0)")
    return r[1]

print("Task 5:"); adf_report(y, 'the original series')
```

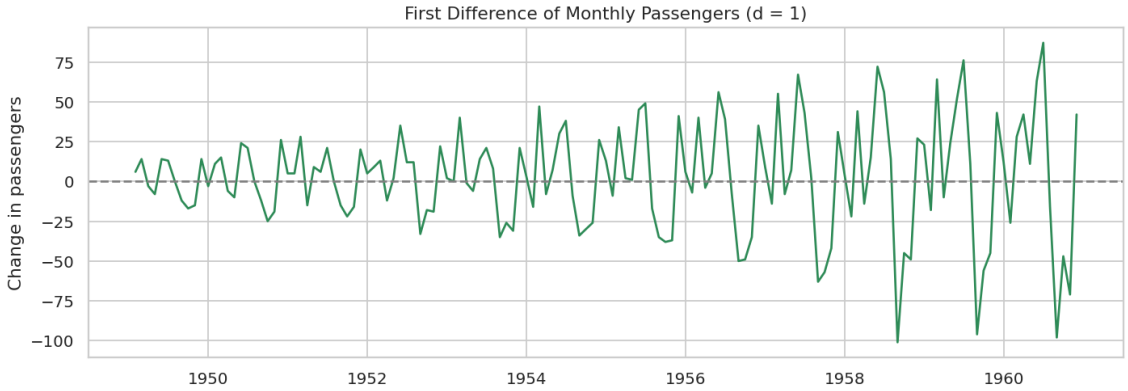
Out [5]:
```
ADF test on the original series
  ADF statistic    : 0.8154
  p-value          : 0.991880
  Lags used        : 13
  Observations     : 130
  Critical value  1%: -3.4817
  Critical value  5%: -2.8840
  Critical value 10%: -2.5788
  Result           : NON-STATIONARY (p >= 0.05, cannot reject H0)
```

Observation: The p-value is 0.9919, far above 0.05, and the ADF statistic (0.8154) is above all critical values, so the null hypothesis of a unit root cannot be rejected. The original series is non-stationary, which agrees with the trend and growing variance seen in Task 4.

Task 6: Make the series stationary by first differencing

In [6]:
```
diff1 = y.diff().dropna()
print("Task 6: first difference (first 5 values):"); print(diff1.head())
plt.figure(figsize=(11, 4)); plt.plot(diff1, color='seagreen'); plt.axhline(0, color='grey', ls='--')
plt.title('First Difference (d = 1)'); plt.tight_layout(); plt.show()
```

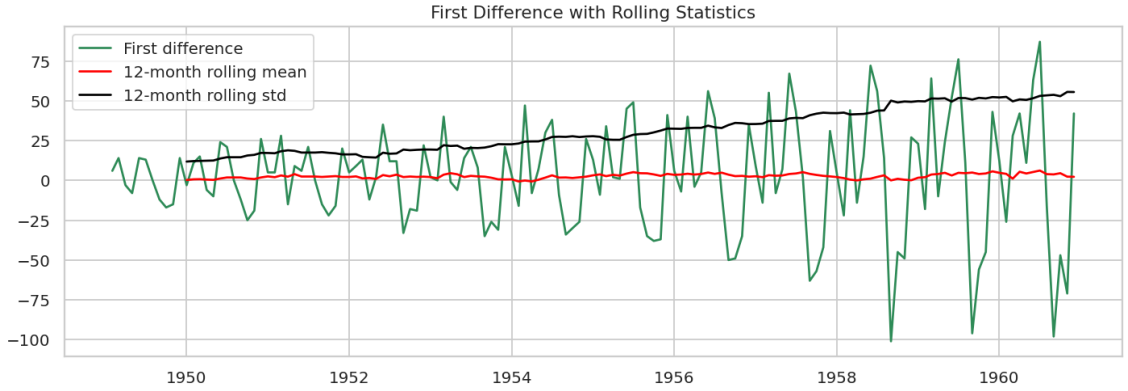
Out [6]:
```
first difference (first 5 values):
Month
1949-02-01     6.0
1949-03-01    14.0
1949-04-01    -3.0
1949-05-01    -8.0
1949-06-01    14.0
Freq: MS
```



Task 7: Re-run ADF on the differenced series and visualise it

In [7]:
```
print("Task 7:"); adf_report(diff1, 'the first difference (d = 1)')
fig, ax = plt.subplots(figsize=(11, 4))
ax.plot(diff1, color='seagreen', label='First difference')
ax.plot(diff1.rolling(12).mean(), color='red', label='12-month rolling mean')
ax.plot(diff1.rolling(12).std(), color='black', label='12-month rolling std')
ax.set_title('First Difference with Rolling Statistics'); ax.legend(); plt.tight_layout(); plt.show()
```

Out [7]:
```
ADF test on the first difference (d = 1)
  ADF statistic    : -2.8293
  p-value          : 0.054213
  Lags used        : 12
  Observations     : 130
  Critical value  1%: -3.4817
  Critical value  5%: -2.8840
  Critical value 10%: -2.5788
  Result           : NON-STATIONARY (p >= 0.05, cannot reject H0)
```



Observation: Differencing once removes the trend: the rolling mean is flat near zero. However, the ADF p-value is 0.0542, just above 0.05, so the series is still (marginally) non-stationary. The rolling standard deviation also keeps rising, so the variance is not constant yet.

Task 8: Second difference or log transformation

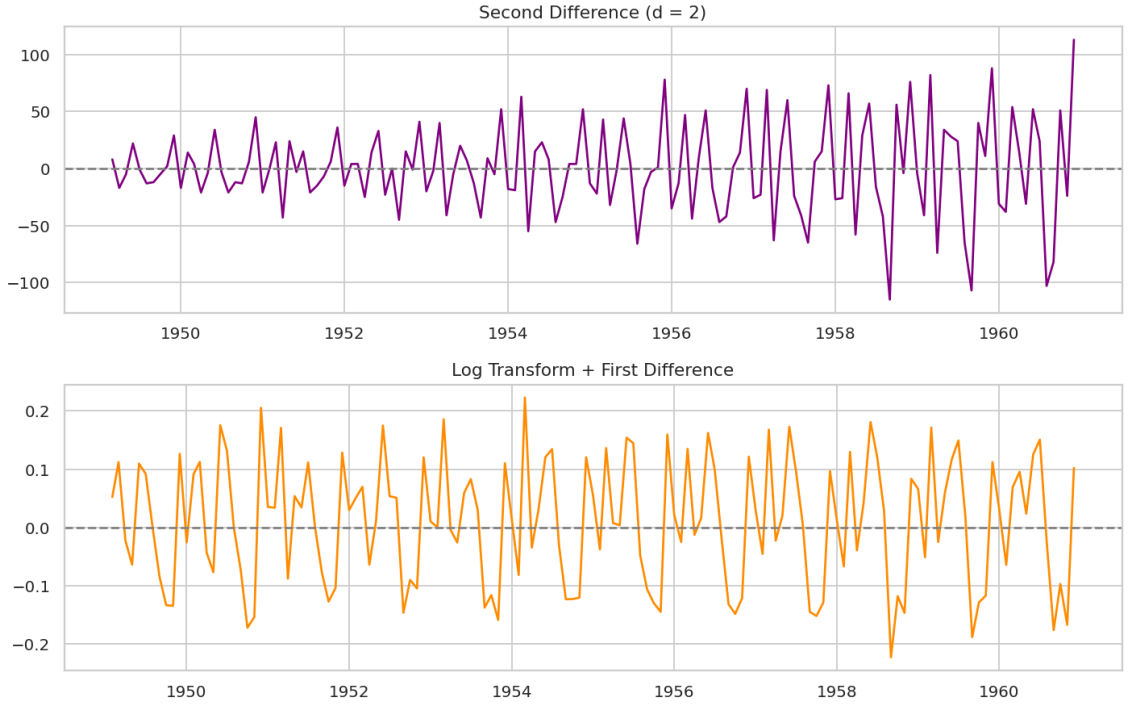
In [8]:

```
print("Task 8: first difference is still non-stationary, so other options are tested.")
diff2 = y.diff().diff().dropna()
adf_report(diff2, 'the second difference (d = 2)')
log_diff = np.log(y).diff().dropna()
adf_report(log_diff, 'the log-transformed series with one difference')
d = 2
print("\nDifferencing order used for ARIMA: d =", d)
fig, ax = plt.subplots(2, 1, figsize=(11, 7))
ax[0].plot(diff2, color='purple'); ax[0].axhline(0, color='grey', ls='--'); ax[0].set_title('Second Difference (d = 2)')
ax[1].plot(log_diff, color='darkorange'); ax[1].axhline(0, color='grey', ls='--'); ax[1].set_title('Log Transform + First Difference')
plt.tight_layout(); plt.show()
```

Out [8]:

```
first difference is still non-stationary, so other options are tested.
ADF test on the second difference (d = 2)
  ADF statistic    : -16.3842
  p-value          : 0.000000
  Lags used        : 11
  Observations     : 130
  Critical value  1%: -3.4817
  Critical value  5%: -2.8840
  Critical value 10%: -2.5788
  Result           : STATIONARY (p < 0.05, reject H0)
ADF test on the log-transformed series with one difference
  ADF statistic    : -2.7171
  p-value          : 0.071121
  Lags used        : 14
  Observations     : 128
  Critical value  1%: -3.4825
  Critical value  5%: -2.8844
  Critical value 10%: -2.5790
  Result           : NON-STATIONARY (p >= 0.05, cannot reject H0)

Differencing order used for ARIMA: d = 2
```



Observation: The second difference gives an ADF statistic of -16.38 and a p-value of 0.0000, so it is stationary, and d = 2 is used for ARIMA. The log transform followed by one difference (p = 0.0711) is still non-stationary. The second-difference plot swings around zero but its spread grows over time, so the model assumes constant variance only approximately.

Task 9: Split into training and testing sets (original series)

In [9]:

```
k = int(len(y) * 0.8)                        # 80% train, 20% test, in time order
train, test = y.iloc[:k], y.iloc[k:]
print(f"Task 9: Train = {len(train)} months ({train.index[0]:%Y-%m} to {train.index[-1]:%Y-%m})")
print(f"        Test  = {len(test)} months ({test.index[0]:%Y-%m} to {test.index[-1]:%Y-%m})")
```

Out [9]:

```
Task 9: Train = 115 months (1949-01 to 1958-07)
        Test  = 29 months (1958-08 to 1960-12)
```

Observation: The split keeps the time order (no shuffling): the first 115 months train the model and the last 29 months test it. The original series is used, as the ARIMA model applies the differencing (d = 2) internally.

Task 10: Fit the ARIMA model on the training set

In [10]:

```
rows = []
for p, q in itertools.product(range(4), range(4)):         # d is fixed at 2 (Task 8)
    try:
        r = ARIMA(train, order=(p, 2, q)).fit(); rows.append((p, 2, q, r.aic, r.bic))
    except Exception: pass
grid = pd.DataFrame(rows, columns=['p', 'd', 'q', 'AIC', 'BIC']).sort_values('AIC').head(5)
print("Task 10: Five best ARIMA orders by AIC:"); print(grid.round(2).to_string(index=False))
best = tuple(int(v) for v in grid.iloc[0][['p', 'd', 'q']])
print("Selected order (p, d, q):", best)
model = ARIMA(train, order=best).fit()
print(model.summary())
```

Out [10]:

```
Task 10: Five best ARIMA orders by AIC:
 p  d  q     AIC     BIC
 1  2  2 1054.89 1065.80
 1  2  3 1056.06 1069.69
 2  2  2 1056.17 1069.81
 0  2  3 1057.37 1068.28
 3  2  2 1057.71 1074.07
Selected order (p, d, q): (1, 2, 2)
                               SARIMAX Results
==============================================================================
Dep. Variable:             Passengers   No. Observations:                  115
Model:                 ARIMA(1, 2, 2)   Log Likelihood                -523.445
Date:                Fri, 09 Oct 2026   AIC                           1054.889
Time:                        05:37:08   BIC                           1065.799
Sample:                    01-01-1949   HQIC                          1059.316
                         - 07-01-1958
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.5137      0.123     -4.192      0.000      -0.754      -0.274
```

```
ma.L1         -0.0843     13.690     -0.006      0.995     -26.916      26.748
ma.L2         -0.9155     12.512     -0.073      0.942     -25.438      23.607
sigma2       590.7316   8099.367      0.073      0.942   -1.53e+04    1.65e+04
===================================================================================
Ljung-Box (L1) (Q):                   0.10   Jarque-Bera (JB):                 2.31
Prob(Q):                              0.75   Prob(JB):                         0.31
Heteroskedasticity (H):               5.07   Skew:                             0.03
Prob(H) (two-sided):                  0.00   Kurtosis:                         2.30
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

Observation: A grid search over p, q = 0 to 3 with d = 2 picks ARIMA(1, 2, 2), which has the lowest AIC (1054.89). The AR(1) coefficient is significant (-0.5137, p < 0.001). The two MA coefficients have very large standard errors, so they are not individually significant. The residual checks are acceptable: Ljung-Box p = 0.75 (no left-over autocorrelation) and Jarque-Bera p = 0.31 (residuals are close to normal).

Task 11: Forecast the test period

In [11]:

```
forecast = model.forecast(steps=len(test))
print("Task 11: Actual vs forecast (first 10 test months):")
print(pd.DataFrame({'Actual': test, 'Forecast': forecast.round(1)}).head(10))
```

Out [11]:

```
Task 11: Actual vs forecast (first 10 test months):
            Actual  Forecast
Month
1958-08-01     505     505.3
1958-09-01     404     503.1
1958-10-01     359     509.4
1958-11-01     310     511.3
1958-12-01     337     515.5
1959-01-01     360     518.5
1959-02-01     342     522.1
1959-03-01     406     525.4
1959-04-01     396     528.9
1959-05-01     420     532.3
```

Observation: The forecast starts close to the actual value (505.3 against 505) but then rises in a straight line, because an ARIMA model with d = 2 and no seasonal part cannot reproduce the yearly up-and-down pattern. By September 1958 the forecast is 503.1 while the actual value is 404.

Task 12: Forecast accuracy - MSE, MAE and RMSE

In [12]:

```
mse  = mean_squared_error(test, forecast)
mae  = mean_absolute_error(test, forecast)
rmse = np.sqrt(mse)
mape = (abs((test - forecast) / test).mean()) * 100
print("Task 12: Forecast accuracy on the test set")
print(f"  MSE  : {mse:.2f}\n  MAE  : {mae:.2f}\n  RMSE : {rmse:.2f}\n  MAPE : {mape:.2f}%")
```

Out [12]:

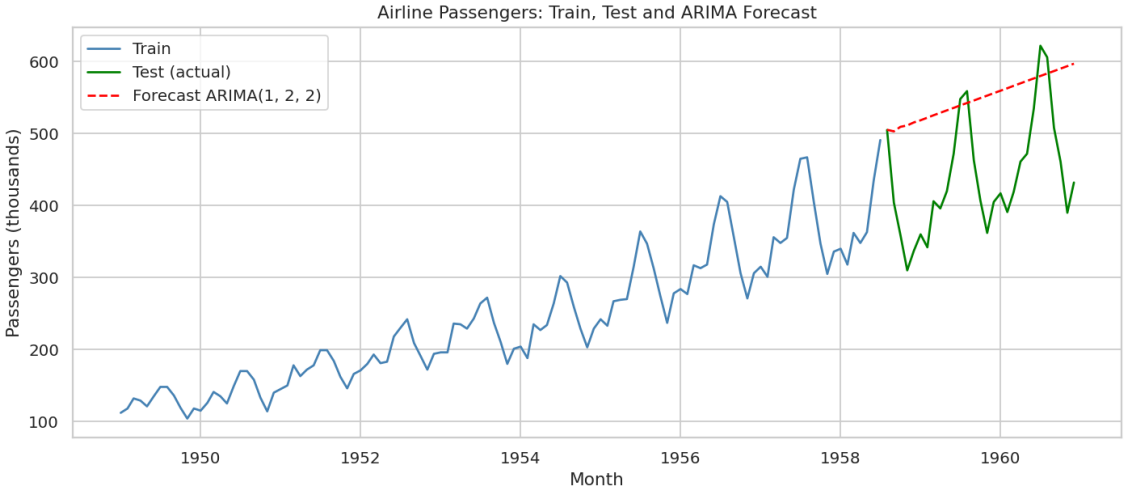
```
Task 12: Forecast accuracy on the test set
  MSE  : 16799.32
  MAE  : 115.32
  RMSE : 129.61
  MAPE : 29.05%
```

Observation: MSE is 16,799.32, MAE is 115.32 and RMSE is 129.61 (thousand passengers), with a mean absolute percentage error of 29.05%. The test months average about 440 thousand passengers, so the typical error of around 115 to 130 (roughly 26% to 29% of that level) is large. The forecast is too high in most months because it keeps the trend going while the actual series dips seasonally.

Task 13: Plot train, test and predictions in a single plot

In [13]:

```
plt.figure(figsize=(11, 5))
plt.plot(train, color='steelblue', label='Train')
plt.plot(test, color='green', label='Test (actual)')
plt.plot(forecast, color='red', ls='--', label=f'Forecast ARIMA{best}')
plt.title('Airline Passengers: Train, Test and ARIMA Forecast')
plt.xlabel('Month'); plt.ylabel('Passengers (thousands)'); plt.legend(); plt.tight_layout(); plt.show()
```



Observation: The red forecast line follows the overall upward direction of the test data but is a smooth line with no seasonal peaks or dips, so it overshoots the troughs (late 1958, early 1959 and late 1960) and only meets the actual values near the summer peaks. A seasonal model such as SARIMA(p, d, q)(P, D, Q)12, or a log transform with seasonal differencing, would capture the yearly pattern better.

Result

Thus a time series analysis and forecast was carried out on the monthly airline passenger data (144 months, 1949-1960). The series showed a rising trend and yearly seasonality (peak in July, low in November) and was non-stationary (ADF p = 0.9919). One difference was not enough (p = 0.0542), while the second difference was stationary (p < 0.0001), so d = 2 was used. An ARIMA(1, 2, 2) model, chosen by lowest AIC, was fitted on the first 115 months and used to forecast the last 29 months, giving MSE 16,799.32, MAE 115.32 and RMSE 129.61. The forecast follows the trend but misses the seasonal swings, so a seasonal ARIMA model is recommended for better accuracy.